```
simplesmtp = require 'simplesmtp'

class Email
  SMTP_PORT = 25
  SMTP_SERVER = 'coffeescriptinaction.com'
  constructor: (options) ->
    ['from', 'to', 'subject', 'body'].forEach (key) =>
      @["_{key}"] = options?[key]
      @[key] = (newValue) ->
        @["_#{key}"] = newValue
        @

  send: ->
    client = simplesmtp.connect SMTP_PORT, SMTP_SERVER
    client.once 'idle', ->
      client.useEnvelope
        from: @_from
        to: @_to
    client.on 'message', ->
      client.write """
      From: "#{@_from}"
      To: #{@_to}
      Subject: #{@_subject}

      #{@_body}

      """
      client.end()
    @


scruffysEmail = new Email()

scruffysEmail
.to('agtron@coffeescriptinaction.com')
.from('scruffy@coffeescriptinaction.com')
.subject('Hi Agtron!')
.body '''

  This is a test email.

'''

scruffysEmail.send()
```

This constructor makes the fluent interface possible by dynamically adding from, to, subject, and body methods to the instance of Email that's created. Each of these methods sets a corresponding variable on the instance when invoked.

Each dynamically added method must return the current object to enable a fluent interface.

When sending the message, properties on the object that were set by invoking the methods are used.

A fluent interface in action: with an instance of Email, the methods calls can be chained.

Finally, CoffeeScript syntax supports a third way to create nice-looking DSLs.

8.2.5. Function passing

The option of omitting parentheses from function calls means that a DSL can be created in CoffeeScript using function composition only (discussed in chapter 6). This has the benefit of allowing a natural-looking DSL made